

DBMS SQL Interview Fundamentals

1. Overview

DBMS SQL interview fundamentals cover the concepts used to design, query, optimize, and safely operate relational databases.

Definition

A **DBMS** stores and manages data. **SQL** is the language used to define, query, update, and control data in relational databases.

This guide focuses on commonly asked DBMS and SQL interview topics:

- Primary key, unique key, and foreign key
- **WHERE** vs **HAVING**
- **INNER JOIN** vs **LEFT JOIN**
- Normalization, 3NF, and BCNF
- Indexing and its read/write trade-off
- B-tree vs B+ tree
- ACID properties
- Isolation levels
- MVCC
- Deadlocks

Why It Matters

Most backend systems depend on databases. A developer must know how to:

- Store data correctly
- Avoid duplicate and inconsistent data
- Write efficient queries
- Design tables properly
- Handle concurrent users safely
- Understand why queries become slow

Where It Is Used In Real Systems

These concepts appear in:

- E-commerce orders and payments
- Banking transactions
- User authentication systems
- Inventory management
- Placement portals
- Social media feeds
- Analytics dashboards
- Backend APIs

Why Interviewers Ask About It

Interviewers ask DBMS SQL questions because they reveal whether you can:

- Design correct database schemas
- Write SQL queries accurately
- Understand data consistency
- Reason about performance
- Handle concurrent access
- Explain practical trade-offs

2. Core Idea

The core idea of DBMS is simple:

Store data in a structured way so that it can be queried, updated, protected, and kept consistent even when many users access it at the same time.

Intuition

Think of a database like a well-managed library:

- Tables are shelves.
- Rows are books.
- Columns are book properties like title, author, and ISBN.
- Keys are labels that uniquely identify or connect books.
- Indexes are catalog cards that help find books quickly.
- Transactions are controlled operations like issuing or returning books.
- Locks and isolation prevent two people from making conflicting changes.

Small Example

Suppose we have a college placement database:

```
Students(student_id, name, email, branch)
Companies(company_id, name)
Applications(application_id, student_id, company_id, status)
```

Here:

- `student_id` can be a primary key in `Students`.
- `email` can be a unique key.
- `student_id` inside `Applications` can be a foreign key referring to `Students`.
- Joins help combine student and company data.
- Indexes help search quickly by `student_id`, `email`, or `company_id`.
- Transactions ensure that updates do not corrupt application records.

Step-By-Step Explanation

1. Design tables to represent real-world entities.
2. Use keys to identify and connect records.
3. Use SQL queries to filter, group, and join data.
4. Normalize tables to reduce redundancy.
5. Add indexes to speed up frequent searches.
6. Use transactions to keep operations reliable.
7. Use isolation and concurrency control to handle multiple users safely.

3. Important Subtopics

3.1 Primary Key

What It Means

A **primary key** uniquely identifies each row in a table.

Example:

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100)  
);
```

Why It Matters

Without a primary key, it becomes difficult to uniquely identify, update, or reference a row.

Example

In a **Students** table, two students may have the same name, but they should not have the same **student_id**.

Common Interview Angle

Interviewers often ask:

Can a primary key contain **NULL**?

Answer: No. A primary key must be unique and not null.

3.2 Unique Key

What It Means

A **unique key** ensures that values in a column or set of columns are unique.

Example:

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    email VARCHAR(100) UNIQUE  
);
```

Why It Matters

It prevents duplicate values in fields like email, phone number, or username.

Example

Two users should not register with the same email.

Common Interview Angle

Interviewers ask:

How is a unique key different from a primary key?

A table can have multiple unique keys, but usually only one primary key.

3.3 Foreign Key

What It Means

A **foreign key** is a column in one table that refers to the primary key of another table.

Example:

```
CREATE TABLE Applications (  
    application_id INT PRIMARY KEY,  
    student_id INT,  
    company_id INT,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id)  
);
```

Why It Matters

It maintains referential integrity.

Example

An application should not exist for a `student_id` that is not present in the `Students` table.

Common Interview Angle

Interviewers may ask:

What happens if a referenced row is deleted?

Depending on constraints, deletion may be restricted, cascaded, or set to null.

3.4 WHERE vs HAVING

What It Means

WHERE filters rows before grouping. **HAVING** filters groups after **GROUP BY**.

Example:

```
SELECT branch, COUNT(*) AS total_students
FROM Students
WHERE active = 1
GROUP BY branch
HAVING COUNT(*) > 10;
```

Why It Matters

Using **WHERE** and **HAVING** correctly affects query correctness and performance.

Example

Find branches having more than 10 active students:

- **WHERE active = 1** first selects active students.
- **GROUP BY branch** groups them.
- **HAVING COUNT(*) > 10** filters branches.

Common Interview Angle

Interviewers ask:

Can aggregate functions be used in **WHERE**?

Usually no. Aggregate filters belong in **HAVING**.

3.5 INNER JOIN vs LEFT JOIN

What It Means

INNER JOIN returns only matching rows from both tables. **LEFT JOIN** returns all rows from the left table and matching rows from the right table.

Example:

```
SELECT s.name, a.status
FROM Students s
```

```
INNER JOIN Applications a
ON s.student_id = a.student_id;
```

```
SELECT s.name, a.status
FROM Students s
LEFT JOIN Applications a
ON s.student_id = a.student_id;
```

Why It Matters

Choosing the wrong join can accidentally remove important rows.

Example

If you want all students, including those who have not applied to any company, use **LEFT JOIN**.

Common Interview Angle

Interviewers ask:

Which join should be used to find students who have not applied?

Use **LEFT JOIN** with **WHERE right_table.column IS NULL**.

```
SELECT s.student_id, s.name
FROM Students s
LEFT JOIN Applications a
ON s.student_id = a.student_id
WHERE a.application_id IS NULL;
```

3.6 Normalization

What It Means

Normalization is the process of organizing data to reduce redundancy and avoid update anomalies.

Why It Matters

It helps prevent:

- Duplicate data
- Inconsistent data
- Insert anomalies
- Update anomalies
- Delete anomalies

Example

Bad design:

```
StudentApplications(student_id, student_name, company_id, company_name)
```

Better design:

```
Students(student_id, student_name)
Companies(company_id, company_name)
Applications(student_id, company_id)
```

Common Interview Angle

Interviewers ask:

Why do we normalize databases?

Answer: To reduce redundancy and improve consistency.

3.7 3NF vs BCNF

What It Means

3NF and **BCNF** are normal forms used to reduce dependency-related redundancy.

In simple terms:

- 3NF removes transitive dependency.
- BCNF is stricter than 3NF.

Why It Matters

They help design clean schemas where facts are stored in the right place.

Example

Suppose:

```
Student(student_id, department_id, department_name)
```

If **department_id** determines **department_name**, then department information should be moved to another table.

Common Interview Angle

Interviewers ask:

Is every BCNF table in 3NF?

Yes. But every 3NF table is not necessarily in BCNF.

3.8 Indexing

What It Means

An **index** is a data structure that helps find rows faster.

Example:

```
CREATE INDEX idx_students_email  
ON Students(email);
```

Why It Matters

Indexes speed up reads, especially searches, joins, sorting, and range queries.

Example

Without an index, the DBMS may scan the whole table to find one email. With an index, it can quickly locate the row.

Common Interview Angle

Interviewers ask:

Why do indexes slow down writes?

Because every insert, update, or delete must also update the index structure.

3.9 B-tree vs B+ Tree

What It Means

B-tree and B+ tree are balanced tree data structures used in indexing.

Why It Matters

They keep search, insertion, and deletion efficient even for huge datasets.

Example

Databases commonly use B+ trees because leaf nodes are linked, making range queries efficient.

Common Interview Angle

Interviewers ask:

Why is B+ tree preferred in databases?

Because all actual data pointers are at leaf nodes, and leaf nodes are linked for fast range scans.

3.10 ACID Properties

What It Means

ACID describes properties of reliable database transactions:

- Atomicity
- Consistency
- Isolation
- Durability

Why It Matters

ACID ensures that database operations remain correct even during failures or concurrent access.

Example

In a bank transfer:

- Debit from account A.
- Credit to account B.

Both must happen, or neither should happen.

Common Interview Angle

Interviewers ask:

Which ACID property ensures all-or-nothing behavior?

Atomicity.

3.11 Isolation Levels

What It Means

Isolation levels define how much one transaction can see changes made by another transaction.

Common levels:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

Why It Matters

They balance correctness and performance.

Example

At low isolation, queries may be faster but can see inconsistent data. At high isolation, consistency improves but concurrency may reduce.

Common Interview Angle

Interviewers ask about:

- Dirty reads
- Non-repeatable reads
- Phantom reads
- Serializable isolation

3.12 MVCC

What It Means

MVCC stands for Multi-Version Concurrency Control. It allows readers and writers to work concurrently by keeping multiple versions of data.

Why It Matters

It improves concurrency by reducing blocking between reads and writes.

Example

If one transaction updates a row, another transaction may still read the older committed version instead of waiting.

Common Interview Angle

Interviewers ask:

How does MVCC improve performance?

It allows non-blocking reads by using versioned rows.

3.13 Deadlocks

What It Means

A **deadlock** occurs when two or more transactions wait forever for each other to release locks.

Why It Matters

Deadlocks can block database progress and must be detected or prevented.

Example

Transaction T1 locks Row A and wants Row B.
Transaction T2 locks Row B and wants Row A.
Both wait for each other.

Common Interview Angle

Interviewers ask:

How can deadlocks be handled?

By detection, timeout, rollback, consistent lock ordering, and shorter transactions.

4. Real-World Example

Consider an online placement portal.

Tables

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100) UNIQUE,  
  branch VARCHAR(20)  
);  
  
CREATE TABLE Companies (  
  company_id INT PRIMARY KEY,  
  name VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE Applications (  
  application_id INT PRIMARY KEY,  
  student_id INT,  
  company_id INT,  
  status VARCHAR(20),  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (company_id) REFERENCES Companies(company_id)  
);
```

How Concepts Apply

- Primary keys uniquely identify students, companies, and applications.
- Unique keys prevent duplicate emails and company names.
- Foreign keys ensure applications refer to valid students and companies.
- **INNER JOIN** finds students who applied.
- **LEFT JOIN** finds students who have not applied.
- Indexes improve lookup by email or company.
- Transactions safely update application status.

- Isolation prevents inconsistent reads during updates.
- Deadlock handling is needed when many recruiters update records simultaneously.

5. Diagrams / Mental Models

Key Relationship

```

Students
-----
student_id  PRIMARY KEY
email       UNIQUE

Applications
-----
application_id PRIMARY KEY
student_id    FOREIGN KEY -> Students.student_id
company_id    FOREIGN KEY -> Companies.company_id

```

WHERE vs HAVING Flow

```

FROM
|
WHERE filters individual rows
|
GROUP BY creates groups
|
HAVING filters groups
|
SELECT
|
ORDER BY

```

INNER JOIN vs LEFT JOIN

```

Students:      A  B  C
Applications:  A  C

INNER JOIN result:
A, C

LEFT JOIN result:
A, B, C
B has NULL application data

```

Index Mental Model

```
Without index:  
Scan row 1 -> row 2 -> row 3 -> ... -> row N  
  
With index:  
Use sorted structure -> jump near required row -> fetch data
```

Deadlock Mental Model

```
T1 holds Lock A ---- waits for ---- Lock B  
T2 holds Lock B ---- waits for ---- Lock A  
  
Result: deadlock
```

6. Common Interview Questions

1. What is the difference between primary key and unique key?

Answer: A primary key uniquely identifies each row and cannot be null. A unique key also enforces uniqueness, but a table can have multiple unique keys.

Expected key points:

- Primary key is the main row identifier.
- Primary key cannot be null.
- Unique key prevents duplicates.
- Multiple unique keys can exist.

Common mistakes:

- Saying primary key and unique key are exactly the same.
- Forgetting the null restriction on primary key.

2. What is a foreign key?

Answer: A foreign key is a column that refers to a key in another table. It maintains referential integrity.

Expected key points:

- Connects two tables.
- Prevents invalid references.
- Can support cascade operations.

Common mistakes:

- Saying foreign key must always be unique.
- Confusing foreign key with primary key.

3. What is the difference between **WHERE** and **HAVING**?

Answer: **WHERE** filters rows before grouping. **HAVING** filters groups after aggregation.

Expected key points:

- **WHERE** works before **GROUP BY**.
- **HAVING** works after **GROUP BY**.
- Aggregate conditions usually go in **HAVING**.

Common mistakes:

- Using aggregate functions directly in **WHERE**.
- Thinking **HAVING** is only a replacement for **WHERE**.

4. What is the difference between **INNER JOIN** and **LEFT JOIN**?

Answer: **INNER JOIN** returns only matching rows from both tables. **LEFT JOIN** returns all rows from the left table and matching rows from the right table, with **NULL** for missing matches.

Expected key points:

- Inner join removes unmatched rows.
- Left join preserves left table rows.
- Left join helps find missing relationships.

Common mistakes:

- Saying left join returns all rows from both tables.
- Forgetting that unmatched right-side columns become **NULL**.

5. What is normalization?

Answer: Normalization is the process of organizing database tables to reduce redundancy and avoid anomalies.

Expected key points:

- Reduces duplicate data.
- Improves consistency.
- Uses normal forms.

Common mistakes:

- Saying normalization always improves performance.
- Ignoring anomalies.

6. What is the difference between 3NF and BCNF?

Answer: 3NF removes transitive dependencies. BCNF is stricter and requires every determinant to be a candidate key.

Expected key points:

- BCNF is stronger than 3NF.

- Every BCNF relation is in 3NF.
- Some 3NF relations may violate BCNF.

Common mistakes:

- Saying 3NF and BCNF are identical.
- Not knowing that BCNF is stricter.

7. Why does indexing speed up reads?

Answer: Indexing speeds up reads because it allows the DBMS to locate rows using a structured search instead of scanning the whole table.

Expected key points:

- Avoids full table scan.
- Helps **WHERE**, joins, sorting, and range queries.
- Often implemented using B+ trees or hash indexes.

Common mistakes:

- Saying indexes always make every query faster.
- Ignoring selectivity.

8. Why do indexes slow down writes?

Answer: Indexes slow down writes because inserts, updates, and deletes must update both the table and the index structures.

Expected key points:

- Extra maintenance cost.
- More storage needed.
- Too many indexes hurt write-heavy systems.

Common mistakes:

- Saying indexes only affect reads.
- Creating indexes on every column.

9. What are ACID properties?

Answer: ACID stands for Atomicity, Consistency, Isolation, and Durability. These properties make transactions reliable.

Expected key points:

- Atomicity means all or nothing.
- Consistency keeps rules valid.
- Isolation controls concurrent transactions.
- Durability survives committed changes.

Common mistakes:

- Confusing consistency with eventual consistency.
- Mixing isolation with durability.

10. What are isolation levels?

Answer: Isolation levels define how much one transaction is isolated from other concurrent transactions.

Expected key points:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable
- Trade-off between consistency and concurrency

Common mistakes:

- Saying higher isolation is always better.
- Not knowing dirty, non-repeatable, and phantom reads.

11. What is MVCC?

Answer: MVCC is a concurrency control method where the database keeps multiple versions of rows so readers can read old committed versions while writers update newer versions.

Expected key points:

- Multi-version concurrency control.
- Improves read/write concurrency.
- Reduces blocking.
- Used in databases like PostgreSQL and MySQL InnoDB.

Common mistakes:

- Saying MVCC means no locks are ever used.
- Ignoring version cleanup.

12. What is a deadlock in DBMS?

Answer: A deadlock occurs when transactions wait for each other in a circular dependency and none can proceed.

Expected key points:

- Circular wait.
- Locks are involved.
- DBMS may abort one transaction.
- Prevention includes consistent lock order.

Common mistakes:

- Confusing deadlock with slow query.

- Saying deadlocks can only happen in operating systems.

7. Deep-Dive Questions

1. Why is B+ tree preferred over B-tree for database indexing?

B+ tree is preferred because all data pointers are stored at leaf nodes, and leaf nodes are linked. This makes range queries efficient.

Example:

```
SELECT *  
FROM Students  
WHERE student_id BETWEEN 100 AND 200;
```

A B+ tree can find the first matching leaf and then scan linked leaves sequentially.

2. Can a foreign key reference a unique key instead of a primary key?

Yes. A foreign key can reference a candidate key, which may be a primary key or a unique key, depending on the DBMS rules.

The referenced column must uniquely identify rows.

3. How can an index be useless even if it exists?

An index may be ignored when:

- The table is very small.
- The column has low selectivity.
- The query applies a function on the indexed column.
- The query returns a large portion of the table.

Example:

```
WHERE LOWER(email) = 'a@example.com'
```

This may not use a normal index on `email`.

4. How does MVCC handle old row versions?

MVCC keeps older versions so active transactions can read consistent snapshots. Later, old versions are cleaned up when no active transaction needs them.

In PostgreSQL, this cleanup is done through vacuuming.

5. How can deadlocks be reduced in application code?

Deadlocks can be reduced by:

- Accessing tables and rows in a consistent order.
- Keeping transactions short.
- Avoiding user interaction inside transactions.
- Updating only required rows.
- Retrying aborted transactions safely.

8. Comparison Tables

Primary Key vs Unique Key vs Foreign Key

Feature	Primary Key	Unique Key	Foreign Key
Purpose	Identifies each row	Prevents duplicate values	Connects tables
Allows duplicate values	No	No	Yes, usually
Allows null	No	DBMS-dependent, often allowed	Yes, if column allows null
Number per table	Usually one	Multiple possible	Multiple possible
Used for relationship	Referenced by other tables	Can also be referenced	References another table
Example	<code>student_id</code>	<code>email</code>	<code>student_id</code> in <code>Applications</code>

WHERE vs HAVING

Feature	WHERE	HAVING
Filters	Rows	Groups
Used before or after grouping	Before <code>GROUP BY</code>	After <code>GROUP BY</code>
Aggregate functions	Usually not allowed	Commonly used
Example	<code>WHERE branch = 'CSE'</code>	<code>HAVING COUNT(*) > 10</code>
Performance	Usually better for row filtering	Used when group filtering is needed

INNER JOIN vs LEFT JOIN

Feature	INNER JOIN	LEFT JOIN
Returns matched rows	Yes	Yes
Returns unmatched left rows	No	Yes

Feature	INNER JOIN	LEFT JOIN
Missing right-side values	Not included	Shown as NULL
Use case	Find students with applications	Find all students, even without applications
Common trap	Losing unmatched rows	Filtering right table in WHERE accidentally converts it to inner join

3NF vs BCNF

Feature	3NF	BCNF
Strictness	Less strict	More strict
Main rule	No transitive dependency on key	Every determinant must be a candidate key
Dependency preservation	Usually easier	May not always preserve dependencies
Redundancy removal	Good	Stronger
Relationship	BCNF implies 3NF	3NF does not always imply BCNF

B-tree vs B+ Tree

Feature	B-tree	B+ Tree
Data storage	Internal and leaf nodes may store data	Data pointers stored at leaf nodes
Leaf node links	Not always linked	Usually linked
Range queries	Less efficient	More efficient
Search path	May end at internal node	Always ends at leaf node
Database indexing	Used sometimes	Commonly preferred

ACID Properties

Property	Meaning	Example
Atomicity	All operations happen or none happen	Debit and credit both succeed or both fail
Consistency	Database rules remain valid	Balance cannot become invalid if constraint prevents it
Isolation	Concurrent transactions do not interfere incorrectly	One user does not see half-finished payment
Durability	Committed data survives failure	Order remains saved after crash

Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Concurrency
Read Uncommitted	Possible	Possible	Possible	Highest
Read Committed	Prevented	Possible	Possible	High
Repeatable Read	Prevented	Prevented	Possible in some DBMS	Medium
Serializable	Prevented	Prevented	Prevented	Lowest

9. Common Mistakes

- Thinking primary key and unique key are exactly the same.
- Assuming foreign keys are always unique.
- Using **HAVING** when **WHERE** is enough.
- Using **INNER JOIN** when unmatched rows are required.
- Forgetting that **LEFT JOIN** can produce **NULL** values.
- Assuming normalization always improves query performance.
- Creating indexes on every column.
- Forgetting that indexes slow down writes.
- Saying B-tree and B+ tree are identical.
- Confusing atomicity with isolation.
- Thinking MVCC means locks are never used.
- Confusing deadlock with general blocking.

10. Edge Cases / Special Cases

Unique Key And NULL

Some DBMSs allow multiple **NULL** values in a unique column because **NULL** means unknown, not equal.

LEFT JOIN With WHERE Clause

This query may accidentally behave like an inner join:

```
SELECT s.name, a.status
FROM Students s
LEFT JOIN Applications a
ON s.student_id = a.student_id
WHERE a.status = 'Selected';
```

Because unmatched rows have **NULL** status, they are removed by the **WHERE** condition.

Index On Low-Selectivity Column

An index on a column like **gender** or **is_active** may not help much if many rows have the same value.

Composite Key Order

For a composite index:

```
CREATE INDEX idx_branch_year  
ON Students(branch, graduation_year);
```

Queries filtering by **branch** can use it well. Queries filtering only by **graduation_year** may not use it efficiently.

Deadlocks Can Happen In Correct Code

Deadlocks do not always mean the logic is wrong. In high-concurrency systems, they can happen naturally and should be handled with retries.

Isolation Behavior Varies By DBMS

The exact behavior of **Repeatable Read** or MVCC may differ between MySQL, PostgreSQL, SQL Server, and Oracle.

11. How to Explain in Interview

DBMS concepts help us design correct and efficient databases. Keys maintain identity and relationships, joins combine data, and clauses like **WHERE** and **HAVING** filter data at different stages. Normalization reduces redundancy, while indexes improve read performance at the cost of slower writes. For reliability, databases use ACID transactions, isolation levels, MVCC, and deadlock handling so that concurrent operations remain correct.

12. Quick Revision Notes

Key Definitions

- **Primary key:** Unique and non-null identifier for a row.
- **Unique key:** Ensures column values are unique.
- **Foreign key:** Connects one table to another.
- **WHERE:** Filters rows before grouping.
- **HAVING:** Filters groups after aggregation.
- **INNER JOIN:** Returns only matching rows.
- **LEFT JOIN:** Returns all left rows and matching right rows.
- **Normalization:** Reduces redundancy and anomalies.
- **3NF:** Removes transitive dependency.
- **BCNF:** Every determinant must be a candidate key.
- **Index:** Speeds reads using an auxiliary data structure.
- **ACID:** Transaction reliability properties.
- **MVCC:** Uses row versions for concurrency.
- **Deadlock:** Circular wait between transactions.

Important Points

- Primary key cannot be null.

- Foreign key values must refer to valid parent records unless null is allowed.
- Use **WHERE** before **GROUP BY**.
- Use **HAVING** for aggregate conditions.
- Use **LEFT JOIN** to keep unmatched left-side rows.
- Indexes are excellent for reads but costly for writes.
- B+ trees are good for range queries.
- Higher isolation improves consistency but may reduce concurrency.

Common Comparisons

- Primary key vs unique key
- Unique key vs foreign key
- **WHERE** vs **HAVING**
- **INNER JOIN** vs **LEFT JOIN**
- 3NF vs BCNF
- B-tree vs B+ tree
- Read Committed vs Serializable

Must-Remember Facts

- Every table should usually have a primary key.
- A foreign key enforces referential integrity.
- **HAVING COUNT(*) > 5** filters groups, not rows.
- Indexes require extra storage.
- MVCC improves concurrency by keeping versions.
- Deadlocks are usually handled by aborting one transaction.

Interview Traps

- Do not say indexes always improve performance.
- Do not say normalization always makes queries faster.
- Do not confuse blocking with deadlock.
- Do not forget null behavior in joins.
- Do not use **WHERE** for aggregate filters.

13. Practice Tasks

Task 1: Create Tables With Keys

Create tables for students, companies, and applications using primary keys, unique keys, and foreign keys.

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    branch VARCHAR(20)  
);
```

Task 2: Write A WHERE Query

Find all active CSE students.

```
SELECT *  
FROM Students  
WHERE branch = 'CSE' AND active = 1;
```

Task 3: Write A HAVING Query

Find branches having more than 50 students.

```
SELECT branch, COUNT(*) AS total_students  
FROM Students  
GROUP BY branch  
HAVING COUNT(*) > 50;
```

Task 4: Practice INNER JOIN

Find students who have applied to at least one company.

```
SELECT DISTINCT s.student_id, s.name  
FROM Students s  
INNER JOIN Applications a  
ON s.student_id = a.student_id;
```

Task 5: Practice LEFT JOIN

Find students who have not applied anywhere.

```
SELECT s.student_id, s.name  
FROM Students s  
LEFT JOIN Applications a  
ON s.student_id = a.student_id  
WHERE a.application_id IS NULL;
```

Task 6: Normalize A Table

Convert this table:

```
Applications(student_id, student_name, company_id, company_name, status)
```

Into:

```
Students(student_id, student_name)
Companies(company_id, company_name)
Applications(student_id, company_id, status)
```

Task 7: Identify Index Candidates

Given frequent queries:

```
SELECT * FROM Students WHERE email = ?;
SELECT * FROM Applications WHERE company_id = ?;
```

Create suitable indexes.

```
CREATE INDEX idx_students_email ON Students(email);
CREATE INDEX idx_applications_company_id ON Applications(company_id);
```

Task 8: Trace A Deadlock

Write two transaction sequences where:

- T1 locks row A, then requests row B.
- T2 locks row B, then requests row A.

Explain why this causes a deadlock.

Task 9: Explain ACID Using Bank Transfer

Explain debit and credit operations using atomicity, consistency, isolation, and durability.

Task 10: Compare Isolation Levels

For each isolation level, identify which anomalies are possible:

- Dirty read
- Non-repeatable read
- Phantom read

14. Final Cheat Sheet

Core Definition

DBMS SQL fundamentals cover how relational databases store, query, relate, optimize, and protect data.

Why It Matters

These concepts are essential for designing reliable backend systems, writing correct SQL queries, and understanding database performance.

Most Asked Questions

- Primary key vs unique key vs foreign key
- WHERE vs HAVING
- INNER JOIN vs LEFT JOIN
- What is normalization?
- 3NF vs BCNF
- Why do indexes speed reads but slow writes?
- B-tree vs B+ tree
- What are ACID properties?
- What are isolation levels?
- What is MVCC?
- What is a deadlock?

Common Comparisons

Comparison	One-Line Difference
Primary key vs unique key	Primary key identifies a row and cannot be null; unique key only prevents duplicates.
Unique key vs foreign key	Unique key enforces uniqueness; foreign key enforces relationship validity.
WHERE vs HAVING	WHERE filters rows; HAVING filters groups.
INNER JOIN vs LEFT JOIN	Inner join keeps matches only; left join keeps all left rows.
3NF vs BCNF	BCNF is stricter than 3NF.
B-tree vs B+ tree	B+ tree stores data pointers at leaves and supports faster range scans.
Read Committed vs Serializable	Read Committed allows more concurrency; Serializable gives strongest isolation.

One-Line Interview Answer

DBMS SQL fundamentals are about designing correct tables using keys and normalization, querying them using filters and joins, improving performance with indexes, and ensuring reliable concurrent transactions using ACID, isolation levels, MVCC, and deadlock handling.